

Foundations of 4D/6D Object Capture for Virtual Environments

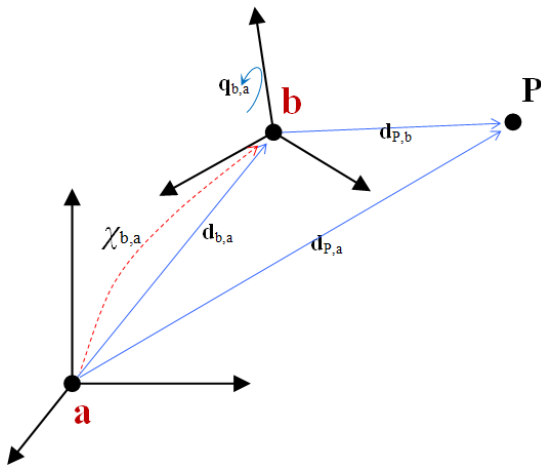


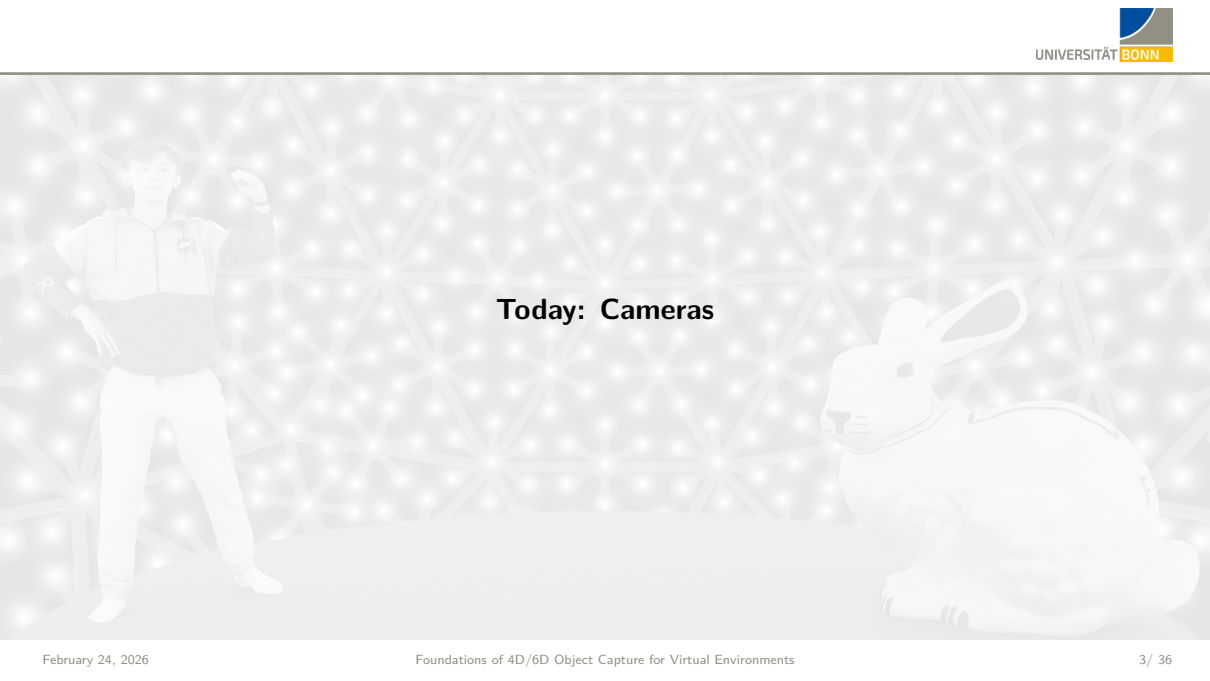
Cameras



February 24, 2026

→ Linear Transformations



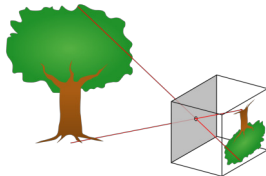


The background of the slide features a virtual environment. On the left, a person stands with one hand on their hip and the other raised. On the right, a white rabbit sits. The background is a repeating pattern of white star-like shapes on a light gray background.

Today: Cameras

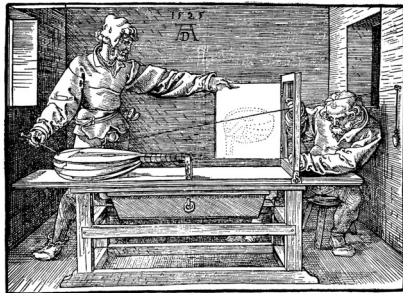
Simple, ideal mathematical model of perspective cameras:

- ▶ No lenses, just one infinitely small pinhole where light passes through
- ▶ No distortion or imperfect optics
- ▶ Image plane behind the pinhole perpendicular to optical axis (line of sight)



→ Concept of perspective imaging already known
500 years ago

Albrecht Dürer: Der Zeichner der Laute (1525)



Task: Derive coordinates of green point $\mathbf{p} = (p_x, p_y)^T \in \mathbb{R}^2$,
i.e. intersection of light ray with image plane

Consider similar triangles and apply the Intercept theorem:

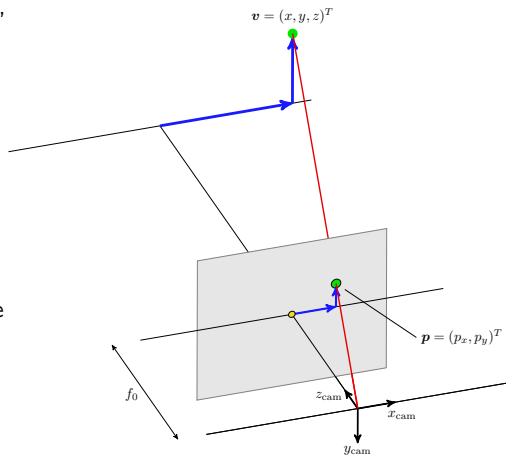
$$\frac{x}{z} = \frac{p_x}{f_0} \quad \Leftrightarrow \quad p_x = \frac{f_0 x}{z}$$

$$\frac{y}{z} = \frac{p_y}{f_0} \quad \Leftrightarrow \quad p_y = \frac{f_0 y}{z}$$

Here, f_0 is the focal length and represents the distance of the image plane to the origin

The intersection point in 3D can thus be written as:

$$(x, y, z)^T \mapsto \left(\frac{f_0 x}{z}, \frac{f_0 y}{z}, f_0 \right)^T$$



The intersection point in 3D can thus be written as:

$$(x, y, z)^T \mapsto \left(\frac{f_0 x}{z}, \frac{f_0 y}{z}, f_0 \right)^T$$

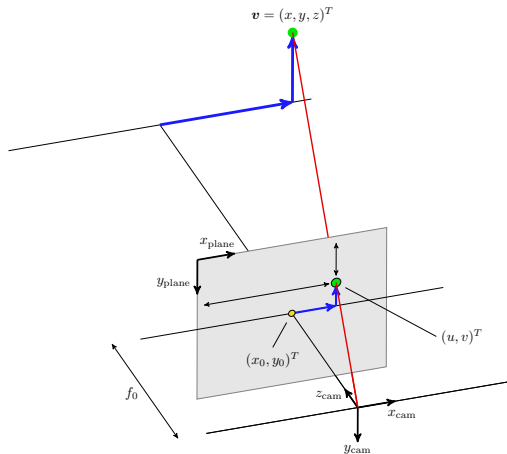
To compute pixel coordinates:

- ▶ Drop z coordinate of the point
- ▶ Shift 2D coordinates by principal point $(x_0, y_0)^T$ such that pixel $(0, 0)^T$ lies on top-left corner

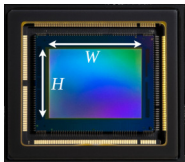
So we get:

$$(u, v)^T := \left(\frac{f_0 x}{z} + x_0, \frac{f_0 y}{z} + y_0 \right)^T$$

→ However, u and v are still defined in the local camera coordinate system (typically in meters)



However, u and v are still defined in the local camera coordinate system (typically in meters)



Scale by pixel density (pixels/meter):

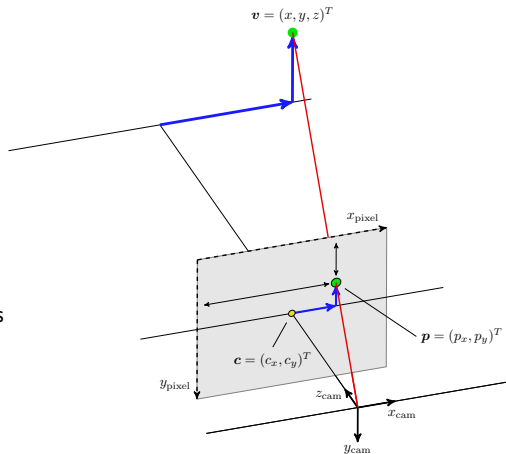
$$\begin{aligned} f_x &= \frac{|\mathcal{P}_x|}{W} \cdot f_0 & c_x &= \frac{|\mathcal{P}_x|}{W} \cdot x_0 \\ f_y &= \frac{|\mathcal{P}_y|}{H} \cdot f_0 & c_y &= \frac{|\mathcal{P}_y|}{H} \cdot y_0 \end{aligned}$$

Finally, the perspective projection (to pixels) can be written as

$$(p_x, p_y)^T = \left(\frac{f_x x}{z} + c_x, \frac{f_y y}{z} + c_y \right)^T$$

Question: Can we write this as a linear transformation?

→ No, since division by z is not linear. But almost.



Definition (Perspective Division)

The operation, mapping a 3D point $\mathbf{v} = (x, y, z)^T \in \mathbb{R}^3$ to the image plane (at $z = 1$), is defined as

$$\pi(x, y, z) = (x/z, y/z)^T$$

If the depth value z is known, this operation can be inverted by

$$\pi_z^{-1}(p_x, p_y) = (p_x, p_y, 1)^T \cdot z$$

Reminder: Homogeneous coordinates:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} x \\ y \end{pmatrix} \mapsto \begin{pmatrix} x \\ y \\ w \end{pmatrix}$$

→ Perspective division can be seen as getting the "same point" at $w = 1$

We can write the perspective projection as a combination of a **linear transformation** and a **perspective division**:

Projection

$$\begin{aligned}(p_x, p_y)^T &= \left(\frac{f_x x}{z} + c_x, \frac{f_y y}{z} + c_y \right)^T \\ &= \pi(\mathbf{K} \cdot (x, y, z)^T)\end{aligned}$$

Unprojection

$$\begin{aligned}(x, y, z)^T &= \left(\frac{p_x - c_x}{f_x} z, \frac{p_y - c_y}{f_y} z, z \right)^T \\ &= \mathbf{K}^{-1} \cdot \pi_z^{-1}(p_x, p_y) \\ &= \mathbf{K}^{-1} \cdot (p_x, p_y, 1)^T \cdot z\end{aligned}$$

→ Inverse operation for known/specified depth z

Here, $\mathbf{K}$ is called **intrinsic camera matrix** and contains the focal lengths $(f_x, f_y)^T$ and principal point $(c_x, c_y)^T$:

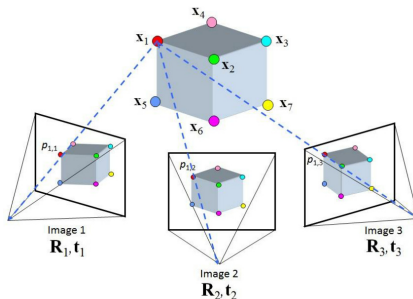
$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

Observation: Up to now, we have assumed that the point lies in front of the camera, i.e. in the local coordinates system of the camera

→ In practice, the scene is defined in world coordinates

$$T = \left[\begin{array}{ccc|c} & R & & t \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

- Orientation: 3 DoF rotation
- Position: 3 DoF translation



However: In practice, infinitely many equally good choices for the world coordinate system

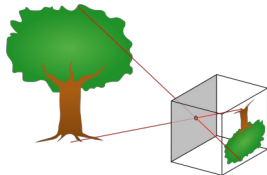
→ Camera poses are typically known only relative to each other

→ Common choice: Pick one camera for world coordinates, e.g. first camera: $R_1 = I$, $t_1 = 0$

How can we map from a 3D point in world coordinates to a 2D pixel?

Solution: Use homogeneous coordinates and chain **Intrinsics** and **Extrinsics**

$$\underbrace{\begin{pmatrix} p_x \\ p_y \end{pmatrix}}_{2 \times 1} = \underbrace{\pi}_{3 \times 1 \rightarrow 2 \times 1} \left(\underbrace{\begin{bmatrix} \textcolor{brown}{K} & \begin{matrix} 0 \\ 0 \\ 0 \end{matrix} \end{bmatrix}}_{3 \times 4} \cdot \underbrace{\begin{bmatrix} \textcolor{green}{R} & \textcolor{green}{t} \\ \hline 0 & 0 & 0 & 1 \end{bmatrix}}_{4 \times 4} \cdot \underbrace{\begin{pmatrix} x_{\text{world}} \\ y_{\text{world}} \\ z_{\text{world}} \\ 1 \end{pmatrix}}_{4 \times 1} \right)$$

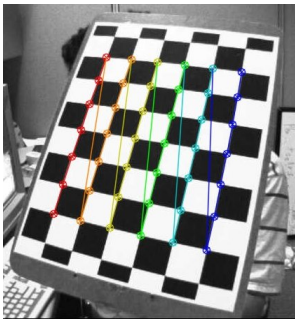


Note:

- So far, we derived the pinhole model in the common context of Computer Vision
→ **OpenCV** notation and convention
- There are similar derivations/formulas if this topic is viewed from the perspective of Computer Graphics
→ **OpenGL** notation and convention

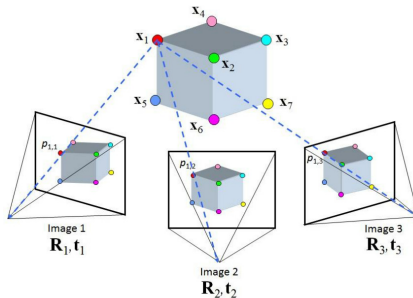
Question: How do we get these camera parameters?

Intrinsics



→ Calibration

Extrinsics



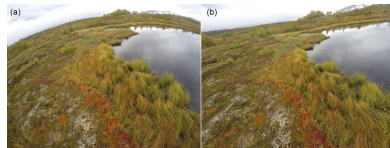
→ Structure from Motion (SfM)

→ Simultaneous Localization and Mapping (SLAM)

Different types of distortion effects:

- ▶ **Geometric Distortion:** Straight lines are bent due to lens
→ Needs to be correct in order to use pinhole model
- ▶ **Chromatic Aberration:** Different focal lengths for wavelengths
→ Mitigated by more complex lenses
- ▶ **Vignetting:** Light falloff around border and corners of image
→ Often automatically corrected in post-processing step

→ How can geometric distortion be corrected?



Motivation: First, we need a model of geometric distortion before correcting for it

→ Define a function from an ideal pixel $p \in \mathbb{R}^2$ to the distorted pixel

$$p_{\text{distorted}} := p + p_{\text{radial}} + p_{\text{tangential}}$$

with the distortion parameters

$$d = \left(\underbrace{k_1 \quad k_2 \quad k_3}_{\text{radial}} \quad \underbrace{p_1 \quad p_2}_{\text{tangential}} \right)^T$$

Note: More complex effects can be expressed with additional parameters

First, transform the pixel into the camera coordinate system by unprojecting to the image plane at $z = 1$:

$$(\tilde{x}, \tilde{y})^T = K^{-1} \cdot (p_x, p_y, 1)^T$$



Radial Distortion: Depends on radius to the optical axis

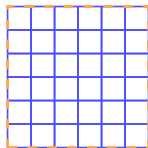
$$r^2 = \tilde{x}^2 + \tilde{y}^2$$

→ Caused by imperfect lens optics

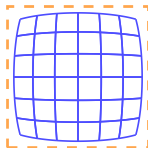
Can be modeled via polynomial function:

$$\tilde{x}_{\text{radial}} = \tilde{x} \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

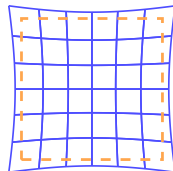
$$\tilde{y}_{\text{radial}} = \tilde{y} \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$



No distortion
 $k_1 = 0, k_2 = 0, k_3 = 0$



Barrel radial distortion
 $k_1 < 0, k_2 = 0, k_3 = 0$



Pincushion radial distortion
 $k_1 > 0, k_2 = 0, k_3 = 0$

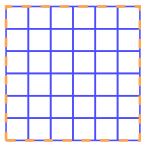
Tangential Distortion: Depends on position (and radius) of the pixel relative to the optical axis

→ Caused by lens being tilted relative to the image plane

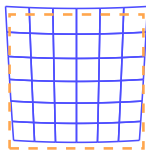
Can be modeled via shift:

$$\tilde{x}_{\text{tangential}} = \tilde{x} + (2 p_1 \tilde{x} \tilde{y} + p_2 (r^2 + 2 \tilde{x}^2))$$

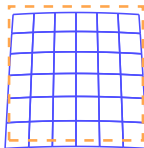
$$\tilde{y}_{\text{tangential}} = \tilde{y} + (p_1 (r^2 + 2 \tilde{y}^2) + 2 p_2 \tilde{x} \tilde{y})$$



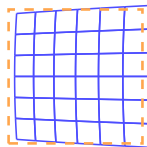
No distortion
 $p_1 = 0, p_2 = 0$



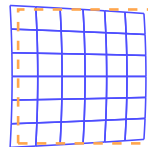
Vertical tangential distortion
 $p_1 > 0, p_2 = 0$



Vertical tangential distortion
 $p_1 < 0, p_2 = 0$

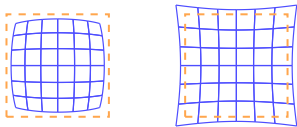


Horizontal tangential distortion
 $p_1 = 0, p_2 > 0$

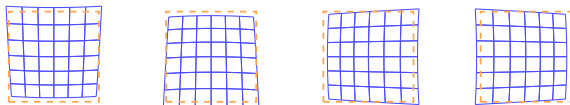


Horizontal tangential distortion
 $p_1 = 0, p_2 < 0$

Radial Distortion



Tangential Distortion



Combination: Thus, the final location of the distorted pixel in camera coordinates is

$$\tilde{x}_{\text{distorted}} = \tilde{x} \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + (2 p_1 \tilde{x} \tilde{y} + p_2 (r^2 + 2 \tilde{x}^2))$$

$$\tilde{y}_{\text{distorted}} = \tilde{y} \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + (p_1 (r^2 + 2 \tilde{y}^2) + 2 p_2 \tilde{x} \tilde{y})$$

By projecting them to pixel coordinates, we finally get the distorted pixel in image space

$$\mathbf{p}_{\text{distorted}} = \pi(\mathbf{K} \cdot (\tilde{x}_{\text{distorted}}, \tilde{y}_{\text{distorted}}, 1)^T)$$

Idea: Use *known* object as calibration target, e.g. checkerboard¹

- ▶ Detect 2D pixel positions $\mathbf{p}_i = (p_{x,i}, p_{y,i})^T \in \mathbb{R}^2$ of target object's pattern
- ▶ Each position has a corresponding 3D point $\mathbf{v}_i = (v_{x,i}, v_{y,i}, v_{z,i})^T \in \mathbb{R}^3$ in the known template
- ▶ Assumption: Planar object lies at $z = 0$
 $\rightarrow v_{z,i} = 0$



So we have the following correspondences:

$$\underbrace{s}_{\text{before } \pi} \cdot \begin{pmatrix} p_{x,i} \\ p_{y,i} \\ 1 \end{pmatrix} = \mathbf{K} \cdot \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix}}_{\text{last row, i.e. } w, \text{ dropped}} \cdot \begin{pmatrix} v_{x,i} \\ v_{y,i} \\ 0 \\ 1 \end{pmatrix} = \mathbf{K} \cdot \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix}}_{3 \times 3} \cdot \begin{pmatrix} v_{x,i} \\ v_{y,i} \\ 1 \end{pmatrix}$$

¹Z. Zhang. "A flexible new technique for camera calibration". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2002).

Question: How many correspondences do we need?

$$\underbrace{s}_{\text{before } \pi} \cdot \begin{pmatrix} p_{x,i} \\ p_{y,i} \\ 1 \end{pmatrix} = \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix}}_{3 \times 3} \cdot \begin{pmatrix} v_{x,i} \\ v_{y,i} \\ 1 \end{pmatrix} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}}_{\mathbf{H}} \cdot \begin{pmatrix} v_{x,i} \\ v_{y,i} \\ 1 \end{pmatrix}$$

► Due to scaling $s \in \mathbb{R}$, the matrix $\mathbf{H}$ has 8 DoF

► Each point gives 2 variables

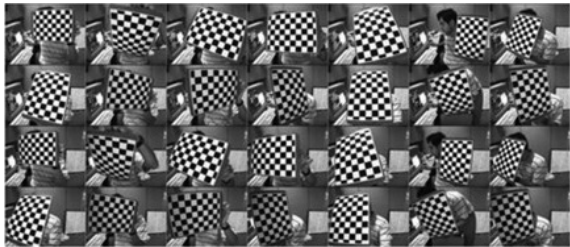
→ ≥ 4 points per image to get $\mathbf{H}$

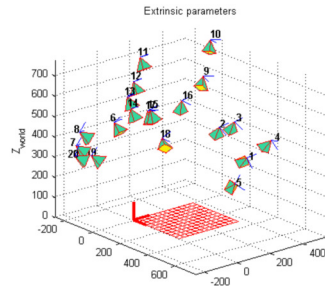
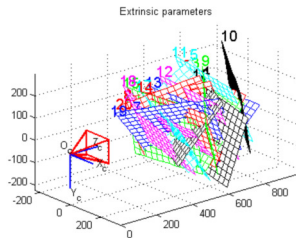
► Extrinsics $[\mathbf{R} | \mathbf{t}]$ has 6 DoF

► Two constraints:

$$\langle \mathbf{r}_1 | \mathbf{r}_2 \rangle = 0 \quad \|\mathbf{r}_1\| = \|\mathbf{r}_2\| = 1$$

→ ≥ 3 images to decompose $\mathbf{H}$





What we can get from calibration images:

Extrinsics $[R | t]$

Intrinsics K

Lens distortion

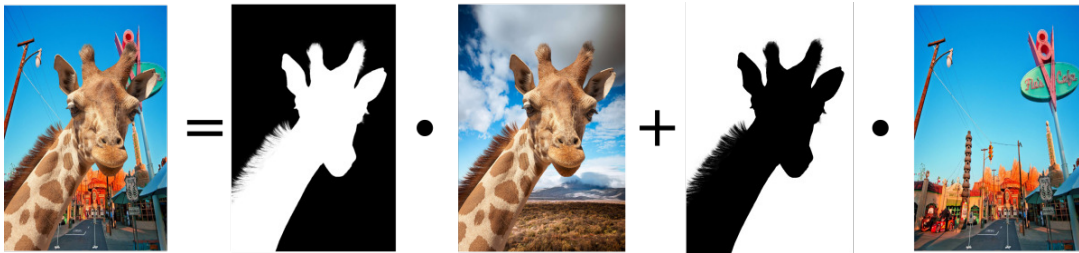
$$d = (k_1, k_2, k_3, p_1, p_2)^T$$

- **Mobile:** Discard
- **Stationary:** Keep, no SfM or SLAM needed

→ Can be found similarly with calibration

For object capturing, we are typically only interested in the foreground

→ Mask image $M(p_x, p_y) \in [0, 1]$



$$I = M \cdot F + (1 - M) \cdot B$$

Types of masks:

- **Segmentation:** *Classification* that assigns each pixel to a specific class

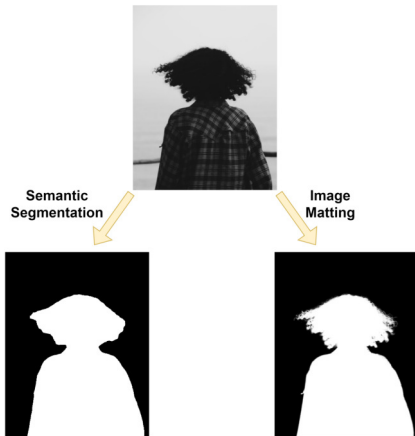
$$M(p_x, p_y) = \begin{cases} 1 & \text{if foreground} \\ 0 & \text{otherwise} \end{cases}$$

- **Matting:** *Regression* that estimates a soft transparency (alpha) value for each pixel

$$M(p_x, p_y) \in [0, 1]$$



→ Non-opaque objects



Problem: How do we know which parts are foreground and which background?

→ Auxiliary data



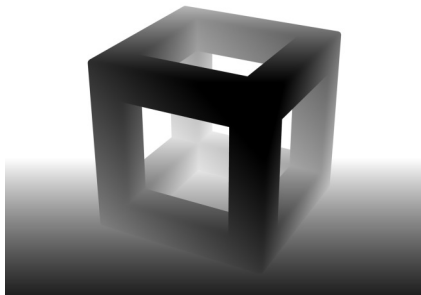
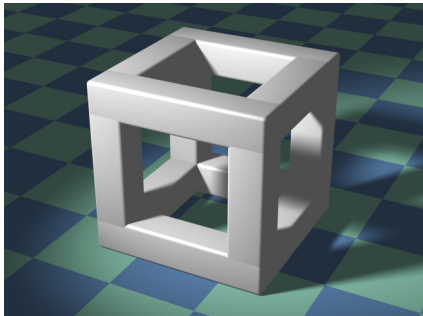
► **Trimap:** Mask with 3 values

$$T_M(p_x, p_y) = \begin{cases} 1 & \text{if certainly foreground} \\ 0 & \text{if certainly background} \\ 0.5 & \text{otherwise (unknown)} \end{cases}$$

► **Background:** RGB image of the background from same pose

► **Click + Scribble:** Annotated sparse set of known foreground/background pixels

A depth map stores how far each pixel is away from the camera as $D(p_x, p_y) \in \mathbb{R}_{>0}$

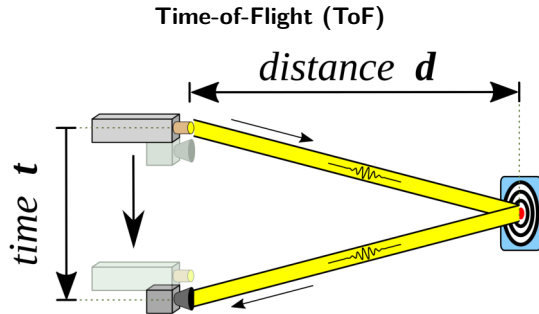
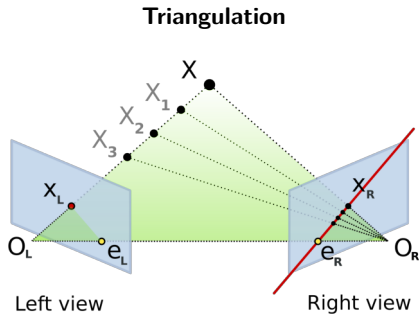


Note: Depth refers to the z coordinate in camera projection, not the actual (Euclidean) distance to the object!

Question: How to capture depth images?

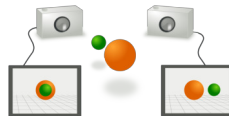
Question: How to capture depth images?

→ There are **two fundamental measurement principles**

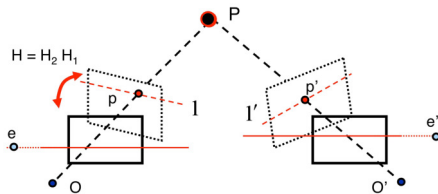
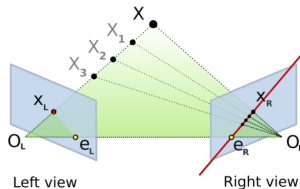


Idea: Observe same point from slightly different positions

- ▶ Match feature from one image to the other image
→ Corresponding pixel must lie on epipolar line
- ▶ Rectify images by re-projecting into common plane parallel to camera base line
→ Sufficient to search along x direction
- ▶ Depth z can be computed as from disparity, focal length, and base line



$$z = \frac{f \cdot (o_l - o_r)}{\underbrace{x_l - x_r}_{\text{disparity}}}$$



Triangulation can be used in several **capturing systems**:

Active / Passive Stereo

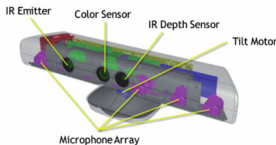
- ▶ Use environment illumination (passive) . . .
- ▶ . . .or emit pattern (active) for more features
- ▶ RGB for passive, IR for active
- ▶ Directly apply triangulation



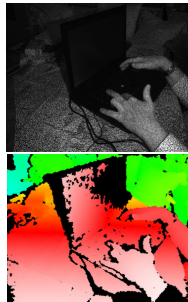
VCI Stereo Rig

Structured Light

- ▶ Active illumination
- ▶ Known IR pattern → Speckle pattern for Kinect
- ▶ Apply triangulation between emitter and camera



Kinect v1



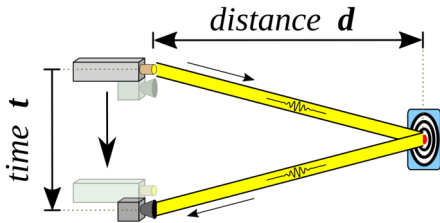
Depth Images: Time of Flight (ToF)

Idea: Emit light into scene and measure time until received by sensor

- ▶ Use pulsed IR emitter to generate distinct signals
- ▶ Employ high-speed detectors to ensure enough precision for time measurements
- ▶ Euclidean distance d along line of sight can be computed by

$$d = \frac{c \cdot \Delta t}{2}$$

→ Depth z depends on angle between line of sight and camera axis

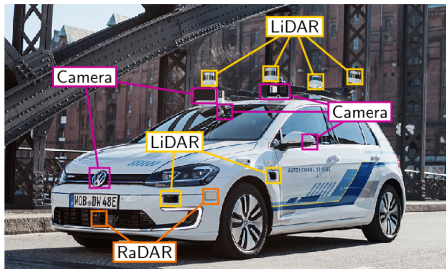
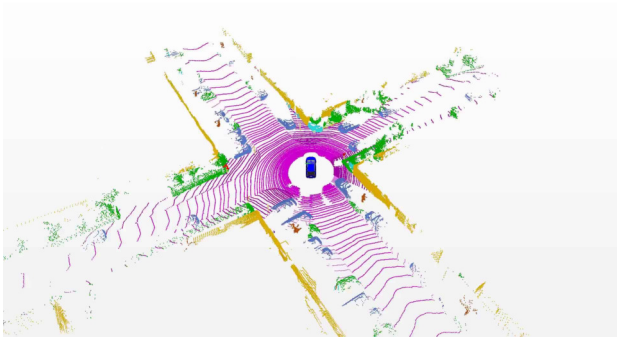


Depth Images: Time of Flight (ToF)

Time of Flight is used in several **capturing systems**:

LiDAR

- ▶ (Usually) rotating laser to scan environment line-by-line
- ▶ Precise but slow



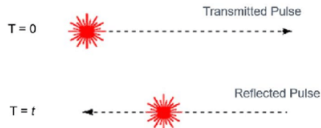
Time of Flight can be used in several **capturing systems**:

Indirect ToF

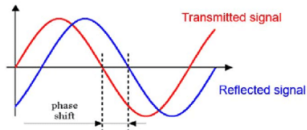
- Emit modulated light $I(t) = I_0 \cdot \sin(2\pi f_m t)$ of frequency f_m

$$d = \frac{c \cdot \Delta\phi}{4\pi f_m}$$

→ Measure phase difference $\Delta\phi$



Time-of-flight



Phase-shift

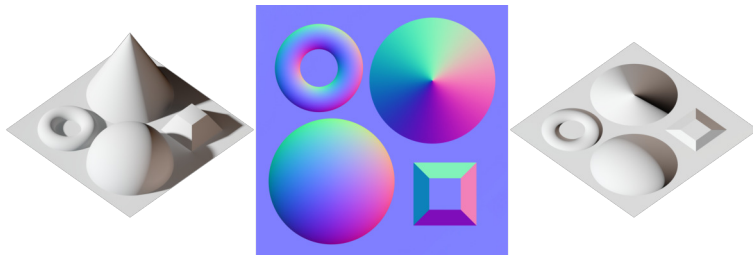


Kinect v2



Azure Kinect

A normal map stores surface orientations as $N(p_x, p_y) \in \mathbb{R}^3$ with $\|N(p_x, p_y)\| = 1$



→ Often used in game engine to approximate surface details for shading

Question: Can we capture normal map images for real-world scenes?

Question: Can we capture normal map images for real-world scenes?²

Idea: Use controlled spherical gradient illumination $L \in [0, 1]$

- ▶ Brightness of illumination L only changes along 1 direction
- ▶ Capture images g_x^+ , g_x^- under positive and negative gradients
- ▶ Full illumination can be "simulated" via the sum $g_x^+ + g_x^-$
- ▶ Assuming diffuse objects, the normal direction component is

$$d_x = \frac{g_x^+ - g_x^-}{g_x^+ + g_x^-}$$

→ This only gives us 1 component



Full Illumination



x-axis Gradient



y-axis Gradient



z-axis Gradient

²W.-C. Ma et al. "Rapid Acquisition of Specular and Diffuse Normal Maps from Polarized Spherical Gradient Illumination". In: *Rendering Techniques* 9.10 (2007).

Idea: Use controlled spherical RGB gradient illumination $\mathbf{L} \in [0, 1]^3$

- ▶ Each color channel encodes gradient along one direction: $(\mathbf{x}, \mathbf{y}, \mathbf{z})^T$
- ▶ Compute normal direction components element-wise and normalize it:

$$\mathbf{d} = \frac{g^+ - g^-}{g^+ + g^-} \quad \Rightarrow \quad \mathbf{N}(p_x, p_y) = \frac{\mathbf{d}(p_x, p_y)}{\|\mathbf{d}(p_x, p_y)\|}$$



Gradient



Inverse Gradient



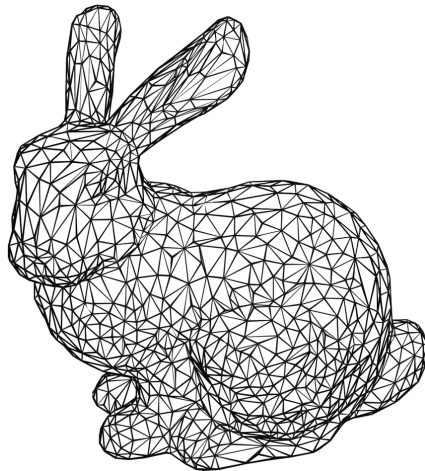
Sum of Gradients



Normals

Next Lecture: Scene Representations

→ **Scene Representations**



- [1] W.-C. Ma, T. Hawkins, P. Peers, C.-F. Chabert, M. Weiss, P. E. Debevec, et al. "Rapid Acquisition of Specular and Diffuse Normal Maps from Polarized Spherical Gradient Illumination". In: *Rendering Techniques* 9.10 (2007).
- [2] Z. Zhang. "A flexible new technique for camera calibration". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2002).